Zewail City of Science and Technology

Communication and Information Engineering Program

CIE 458

Fundamentals of Artificial Intelligence

Game-Playing Agent Project

Part 1: Search-Based Agent for FrozenLake

Project Type	Group Project
Selected Game	FrozenLake
Agent Type	Search-Based Goal Agent
Implemented Algorithms	BFS, UCS, DFS, DLS, IDS, GBFS, A*
Programming Language	Python
GUI Library	Tkinter

Submitted by:

Raed Asham 202200272

Asmaa Sapry 202201338

Zeina Mohamed 202300450

Submitted to:

A. Prof. Ahmed Magdy Mohamed

May 12, 2026

Contents

1 Introduction **2**

2 Task 1: Describe the Selected Game **2**
- 2.1 Selected Game . . . 2
- 2.2 Agent and Environment . . . 3
- 2.3 State-Space Formulation . . . 3
- 2.4 Available Actions . . . 4
- 2.5 Winning and Losing Conditions . . . 4
- 2.6 Reward and Score System . . . 4

3 Task 2: Design the Heuristic Rules **5**
- 3.1 General Search Rules . . . 5
- 3.2 Implemented Search Strategies . . . 5
- 3.3 Uninformed Search Rules . . . 5
  - 3.3.1 Breadth-First Search . . . 6
  - 3.3.2 Uniform-Cost Search . . . 6
  - 3.3.3 Depth-First Search . . . 6
  - 3.3.4 Depth-Limited Search . . . 6
  - 3.3.5 Iterative Deepening Search . . . 6
- 3.4 Informed Search Rules . . . 6
  - 3.4.1 Greedy Best-First Search . . . 7
  - 3.4.2 A* Search . . . 7

4 Task 3: Write the Heuristic/Search Algorithm **7**
- 4.1 General Search Algorithm . . . 7
- 4.2 BFS Pseudocode . . . 8
- 4.3 DFS Pseudocode . . . 8
- 4.4 UCS Pseudocode . . . 9
- 4.5 GBFS Pseudocode . . . 9
- 4.6 A* Pseudocode . . . 9

5 Task 4: Implement and Test the Agent **10**
- 5.1 Implementation Description . . . 10
- 5.2 GUI Visualization . . . 11
- 5.3 Importance of GUI Visualization . . . 11

6 Task 5: Report and Discuss the Results **11**
- 6.1 Testing Setup . . . 11
- 6.2 Results Table . . . 13
- 6.3 Discussion of Results . . . 13
- 6.4 Comparison Between Uninformed and Informed Search . . . 14
- 6.5 Performance Summary . . . 14

7 Conclusion **15**

8 References **15**

1 Introduction

This project implements an intelligent search-based agent for the **FrozenLake** game. The main idea is to model the game as a **state-space search problem**, where the agent starts from an initial state and searches for a sequence of valid actions that leads to the goal state.

The project is related to the main topics covered in the CIE 458 course, especially intelligent agents, state-space problem formulation, uninformed search, informed search, and heuristic search. The course lectures describe search as the process of building agents that can plan ahead to solve problems with many possible states. In this project, the agent plans its path before moving, then follows the discovered safe path from the start cell to the goal cell.

The implemented system contains a graphical user interface that allows the user to press an icon for a specific search strategy. After pressing the icon, the selected algorithm runs automatically, shows the explored states, highlights the final path, and moves the agent from the start to the goal.

2 Task 1: Describe the Selected Game

2.1 Selected Game

The selected game is **FrozenLake**, a discrete grid-based game. The environment is represented as a 4×4 grid. The agent starts from the start cell and must reach the goal cell while avoiding dangerous hole cells.

The map used in this project is:

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

Where:

- **S**: Start state.
- **F**: Frozen safe state.
- **H**: Hole or failure state.
- **G**: Goal state.

The game is suitable for the project because it is discrete, fully observable, deterministic, sequential, static, and single-agent. These assumptions match the type of search problems discussed in the course lectures.

2.2 Agent and Environment

According to the intelligent agent concept, an agent perceives its environment and acts upon it. In this project, the agent perceives its current position on the FrozenLake grid and acts by choosing one of the available movement actions.

The project can be described using the PEAS framework as shown in Table 1.

Table 1: PEAS description of the FrozenLake agent.

PEAS Element	**FrozenLake Project Description**
Performance Measure	Reaching the goal, avoiding holes, minimizing path cost, and reducing explored states.
Environment	A 4×4 FrozenLake grid.
Actuators	Movement actions: Up, Down, Left, and Right.
Sensors	Current state location and map information.

2.3 State-Space Formulation

The FrozenLake game was formulated using the five main components of a state-space problem: initial state, actions, transition model, goal test, and path cost.

Table 2: State-space formulation of FrozenLake.

Component	**Description**
Initial State	State 0, the start cell S.
Actions	Up, Down, Left, and Right.
Transition Model	Applying an action moves the agent to a neighboring valid cell.
Goal Test	Check whether the current state is state 15.
Path Cost	Each valid movement has cost equal to 1.

The states are numbered from 0 to 15 as follows:

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

The hole states are:

$$H = \{5, 7, 11, 12\}$$

The goal state is:

$$G = 15$$

2.4 Available Actions

The agent has four possible discrete actions. These actions are shown in Table 3.

Table 3: Available actions in the FrozenLake environment.

Action	Effect
Up	Move one cell upward.
Down	Move one cell downward.
Left	Move one cell to the left.
Right	Move one cell to the right.

Invalid movements outside the grid are ignored. Movements into holes are avoided by the search algorithm.

2.5 Winning and Losing Conditions

The winning condition occurs when the agent reaches the goal state:

$$\text{Goal State} = 15$$

The losing condition occurs if the agent moves into one of the hole states:

$$\text{Hole States} = \{5, 7, 11, 12\}$$

In the implemented search model, hole states are treated as blocked states, so they are not added to the frontier.

2.6 Reward and Score System

The reward system is simple. Reaching the goal gives a reward of $+1$, while normal movements give a reward of 0. Falling into a hole leads to failure.

Table 4: Reward system of FrozenLake.

Event	Reward
Reaching the goal	$+1$
Moving to a frozen tile	0
Falling into a hole	0 and failure

For the search comparison, the main performance measures are:

- Path length.
- Total path cost.
- Number of explored states.
- Success or failure.

3 Task 2: Design the Heuristic Rules

3.1 General Search Rules

The agent follows general search rules based on the state-space formulation. The search strategy controls how the agent chooses the next node from the frontier.

The general rules used by the agent are:

1. Start from the initial state.
2. Add the initial state to the frontier.
3. Select the next state from the frontier according to the chosen search strategy.
4. Add the selected state to the explored list.
5. Test whether the selected state is the goal.
6. If it is the goal, reconstruct the solution path.
7. If it is not the goal, expand the state.
8. Generate valid neighboring states.
9. Ignore states outside the grid.
10. Ignore hole states.
11. Add valid states to the frontier.
12. Repeat until the goal is found or no states remain.

3.2 Implemented Search Strategies

The project implements both uninformed and informed search strategies:

- Breadth-First Search (BFS).
- Uniform-Cost Search (UCS).
- Depth-First Search (DFS).
- Depth-Limited Search (DLS).
- Iterative Deepening Search (IDS).
- Greedy Best-First Search (GBFS).
- A* Search.

3.3 Uninformed Search Rules

Uninformed search uses only the information available from the problem formulation. It does not use additional knowledge about how close a state is to the goal.

3.3.1 Breadth-First Search

Breadth-First Search expands the shallowest unexpanded node first. In implementation, the frontier is handled as a FIFO queue. New successors are added to the end of the queue. Since all step costs are equal in this project, BFS can find a shortest path.

3.3.2 Uniform-Cost Search

Uniform-Cost Search expands the node with the lowest path cost. Its evaluation function is:

$$f(n) = g(n)$$

where $g(n)$ is the cost from the start state to the current state. In this project, every step has cost 1, so UCS behaves similarly to BFS.

3.3.3 Depth-First Search

Depth-First Search expands the deepest unexpanded node first. In implementation, the frontier is handled as a LIFO stack. DFS may find a valid path, but it does not always guarantee the shortest path.

3.3.4 Depth-Limited Search

Depth-Limited Search is DFS with a maximum depth limit. In this project, the depth limit was set to 6 because the shortest safe path from the start to the goal requires 6 movements.

$$\text{Depth Limit} = 6$$

3.3.5 Iterative Deepening Search

Iterative Deepening Search repeatedly applies depth-limited search using increasing depth limits. It starts with a small depth and increases the limit until the goal is found.

3.4 Informed Search Rules

Informed search uses additional knowledge to expand better nodes earlier. This project uses the Manhattan distance heuristic:

$$h(n) = |r_n - r_g| + |c_n - c_g|$$

where:

- r_n is the row of the current state.
- c_n is the column of the current state.
- r_g is the row of the goal state.
- c_g is the column of the goal state.

3.4.1 Greedy Best-First Search

Greedy Best-First Search expands the node that appears closest to the goal according to the heuristic. Its evaluation function is:

$$f(n) = h(n)$$

It is goal-directed, but it does not consider the actual path cost from the start.

3.4.2 A* Search

A* Search combines the actual path cost and the heuristic estimate. Its evaluation function is:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the cost from the start state to the current state.
- $h(n)$ is the estimated cost from the current state to the goal.

In this project:

$$g(n) = \text{number of steps from the start}$$

$$h(n) = \text{Manhattan distance to the goal}$$

4 Task 3: Write the Heuristic/Search Algorithm

4.1 General Search Algorithm

The general search algorithm used in the project is shown below.

Listing 1: General search algorithm pseudocode.

```
Define the FrozenLake grid
Define the start state
Define the goal state
Define the hole states

Choose a search strategy

Initialize the frontier with the start state
Initialize an empty explored list
Initialize a parent dictionary to store the path

While the frontier is not empty:

    Select a state from the frontier according to the selected
       strategy

```

```
    Add the selected state to the explored list

    If the selected state is the goal:
        Reconstruct the path from goal to start using the parent
           dictionary
        Reverse the path
        Return the path, actions, and explored states

    Generate neighboring states:
        Up
        Down
        Left
        Right

    For each neighbor:
        If the neighbor is inside the grid and not a hole:
            Add it to the frontier if it has not been visited

If the frontier becomes empty:
    Return failure
```

4.2 BFS Pseudocode

Listing 2: Breadth-First Search pseudocode.

```
Create a FIFO queue
Add the start state to the queue

While the queue is not empty:
    Remove the first state from the queue

    If it is the goal:
        Return the path

    Expand the state
    Add all valid unvisited neighbors to the end of the queue
```

4.3 DFS Pseudocode

Listing 3: Depth-First Search pseudocode.

```
Create a LIFO stack
Add the start state to the stack

While the stack is not empty:
    Remove the last state from the stack

    If it is the goal:
        Return the path
```

```
    Expand the state
    Add all valid unvisited neighbors to the stack
```

4.4 UCS Pseudocode

Listing 4: Uniform-Cost Search pseudocode.

```
Create a priority queue
Add the start state with cost 0

While the priority queue is not empty:
    Remove the state with the lowest path cost g(n)

    If it is the goal:
        Return the path

    Expand the state

    For each valid neighbor:
        Calculate the new path cost
        Add or update the neighbor in the priority queue
```

4.5 GBFS Pseudocode

Listing 5: Greedy Best-First Search pseudocode.

```
Create a priority queue
Add the start state using h(start)

While the priority queue is not empty:
    Remove the state with the lowest heuristic value h(n)

    If it is the goal:
        Return the path

    Expand the state

    For each valid neighbor:
        Calculate h(neighbor)
        Add the neighbor to the priority queue
```

4.6 A* Pseudocode

Listing 6: A* Search pseudocode.

```
Create a priority queue

```

```
Set g(start) = 0
Calculate h(start)
Calculate f(start) = g(start) + h(start)

Add the start state to the priority queue

While the priority queue is not empty:
    Remove the state with the lowest f(n)

    If it is the goal:
        Return the path

    Expand the state

    For each valid neighbor:
        Calculate new g(n)
        Calculate h(n)
        Calculate f(n) = g(n) + h(n)

        If this path is better:
            Update the parent
            Add the neighbor to the priority queue
```

5 Task 4: Implement and Test the Agent

5.1 Implementation Description

The project was implemented using **Python** and **Tkinter GUI**. The GUI contains icon buttons for each search strategy:

Icon/Button	Algorithm
BFS	Breadth-First Search
UCS	Uniform-Cost Search
DFS	Depth-First Search
DLS	Depth-Limited Search
IDS	Iterative Deepening Search
GBFS	Greedy Best-First Search
A*	A* Search

When the user clicks an algorithm icon, the agent automatically applies the selected search strategy. The GUI shows three main stages:

1. Explored states.
2. Final path.
3. Agent movement from start to goal.

5.2 GUI Visualization

The visualization uses colors to make the search behavior clear, as shown in Table 5.

Table 5: GUI color meaning.

Color	**Meaning**
Light blue	Frozen safe tile.
Blue	Hole.
Purple	Explored state.
Green	Final solution path.
Red circle	Agent.
Yellow	Goal.

The implementation displays the following information:

- Algorithm name.
- Short explanation of the algorithm.
- Path states.
- Actions.
- Path length.
- Total cost.
- Explored states.
- Number of explored states.

5.3 Importance of GUI Visualization

The GUI helps demonstrate the difference between search strategies. Instead of only printing the solution path, the program visually shows how each strategy expands states before finding the solution. This supports the course concepts of frontier states, explored states, and unexplored states.

6 Task 5: Report and Discuss the Results

6.1 Testing Setup

The algorithms were tested on the same 4×4 FrozenLake map:

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

The movement cost was set as:

$$\text{Each step cost} = 1$$

The agent avoids the hole states:

$$\{5, 7, 11, 12\}$$

The goal state is:

$$15$$

The path found by the implemented algorithms is:

$$0 \rightarrow 4 \rightarrow 8 \rightarrow 9 \rightarrow 13 \rightarrow 14 \rightarrow 15$$

The corresponding actions are:

$$\text{Down} \rightarrow \text{Down} \rightarrow \text{Right} \rightarrow \text{Down} \rightarrow \text{Right} \rightarrow \text{Right}$$

The path length is:

$$6 \text{ steps}$$

The total cost is:

$$6$$

6.2 Results Table

Table 6: Comparison of search strategy results.

Algorithm	Path Found	Path Length	Total Cost	Explored States	Comment
BFS	Yes	6	6	12	Finds the shortest path when all costs are equal.
UCS	Yes	6	6	12	Same result as BFS because all step costs are equal.
DFS	Yes	6	6	7	Found a good path in this run, but not always guaranteed generally.
DLS	Yes	6	6	7	Works because the depth limit was set to 6.
IDS	Yes	6	6	51	Finds the solution but repeats work at each depth level.
GBFS	Yes	6	6	7	Uses Manhattan distance to move toward the goal.
A*	Yes	6	6	12	Uses both path cost and heuristic distance.

6.3 Discussion of Results

The results show that all implemented search strategies successfully found a safe path from the start state to the goal state.

BFS found the shortest path because it explores the search tree level by level. Since all step costs are equal in this FrozenLake map, the first goal found by BFS is a shortest solution.

UCS produced the same path and number of explored states as BFS because all movements have equal cost. If different movement costs were introduced, UCS would become more useful because it selects the path with the lowest accumulated cost.

DFS explored fewer states in this specific implementation because the selected action order guided it quickly toward the safe path. However, DFS is sensitive to action order and does not generally guarantee the shortest path.

DLS found the solution because the depth limit was set to 6, which is enough to reach the goal using the shortest safe path. If the limit were smaller than 6, DLS would fail to reach the goal.

IDS found the same solution but explored more states overall because it repeatedly runs depth-limited search with increasing depth limits. This causes repeated exploration, but it is useful when the solution depth is unknown.

GBFS used Manhattan distance to move toward the goal more directly. It explored fewer states in this map because the heuristic guided it toward promising states. However, GBFS is not guaranteed to be optimal because it uses only $h(n)$ and ignores the actual path cost $g(n)$.

A* used both path cost and heuristic distance:

$$f(n) = g(n) + h(n)$$

This makes A* more balanced than GBFS because it considers both the travelled cost and the estimated remaining distance to the goal.

6.4 Comparison Between Uninformed and Informed Search

Table 7: Comparison between uninformed and informed search.

Type	**Algorithms**	**Main Idea**
Uninformed Search	BFS, UCS, DFS, DLS, IDS	Uses only the problem formulation.
Informed Search	GBFS, A*	Uses heuristic information to guide the search.

Uninformed search is systematic but may expand unnecessary states. Informed search improves the search process by using additional knowledge, such as Manhattan distance, to estimate closeness to the goal.

6.5 Performance Summary

The best results in this specific map were obtained by DFS, DLS, and GBFS in terms of explored states. However, this does not mean that they are always the best algorithms. Their performance depends on action order, depth limit, and heuristic quality.

The most reliable algorithms are:

- **BFS**: reliable for shortest path when all costs are equal.
- **UCS**: reliable for lowest-cost path when costs are different.
- **A***: efficient and balanced when a good heuristic is available.

The most visually useful algorithms for comparison are:

- **BFS**: shows level-by-level exploration.
- **DFS**: shows deep branch exploration.
- **IDS**: shows repeated depth-limited exploration.
- **GBFS**: shows heuristic-guided movement.
- **A***: shows cost and heuristic balanced search.

7 Conclusion

This project successfully models the FrozenLake game as a state-space AI search problem. The agent is a goal-based intelligent agent that perceives its current state and selects actions based on a selected search strategy.

The game was solved using both uninformed and informed search algorithms, including BFS, UCS, DFS, DLS, IDS, GBFS, and A*. The GUI implementation makes the comparison between algorithms clear by showing explored states, final paths, and automatic agent movement.

Overall, A* is the most theoretically balanced search strategy because it uses both the actual path cost and the estimated distance to the goal. BFS and UCS are reliable for optimal solutions in equal-cost or cost-based problems. DFS and DLS can be faster in some cases but are less reliable for guaranteed optimality. GBFS is efficient and goal-directed but depends strongly on the quality of the heuristic.

8 References

1. A. Prof. Ahmed Magdy Mohamed, *CIE 458: Fundamentals of Artificial Intelligence, Lecture 0: Introduction*, Zewail City of Science and Technology, 2026.

2. A. Prof. Ahmed Magdy Mohamed, *CIE 458: Fundamentals of Artificial Intelligence, Lecture 1: Introduction*, Zewail City of Science and Technology, 2026.

3. A. Prof. Ahmed Magdy Mohamed, *CIE 458: Fundamentals of Artificial Intelligence, Lecture 2: Intelligent Agents*, Zewail City of Science and Technology, 2026.

4. A. Prof. Ahmed Magdy Mohamed, *CIE 458: Fundamentals of Artificial Intelligence, Lecture 4: Search Strategies*, Zewail City of Science and Technology, 2026.

5. A. Prof. Ahmed Magdy Mohamed, *CIE 458: Fundamentals of Artificial Intelligence, Lecture 5: Search Strategies Part II*, Zewail City of Science and Technology, 2026.

6. A. Prof. Ahmed Magdy Mohamed, *CIE 458: Fundamentals of Artificial Intelligence, Lecture 6: Search Strategies Part III*, Zewail City of Science and Technology, 2026.

7. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed., Prentice Hall, 2010.

8. Gymnasium Documentation, *FrozenLake Environment*.